

# 《软件的自然性》看软件漏洞挖掘的新视角

## 一、引言

在计算机科学的传统认知中，编程语言被视为一种人造的、形式化的逻辑体系，其核心在于精确的语法结构和严格的语义规则。因此，长久以来，软件工程的研究工具多依赖于形式化方法、静态分析或基于规则的逻辑推理。然而，Abram Hindle 等人发表的经典《On the Naturalness of Software》（软件的自然性）对这一传统观念发起了挑战。该文提出了一个具有范式转移意义的假说：软件不仅是机器指令的集合，更是人类沟通与创作的产物。既然软件是由人类编写的，它必然受到人类认知能力、工作习惯以及沟通模式的深刻约束。这种约束使得软件代码在统计特性上表现出与人类自然语言（如英语）极高的相似性，即具备显著的重复性（Repetitiveness）和可预测性（Predictability）。

本文将对该论文的核心理论、方法论证及实验结果进行详尽的梳理，并以此为基础，深入探讨这一“自然性”假说如何为当前的软件漏洞分析与挖掘任务提供全新的切入点和理论支撑。

## 二、论文核心内容与方法论证

论文开篇即建立了一个核心论点：虽然编程语言在理论上允许构建无限复杂、灵活多变的程序，但在实际的软件开发实践中，程序员编写的代码（Real Software）往往倾向于使用简单的、重复的模式。这种现象的根源在于人类编写代码时的“自然”倾向——为了便于理解和维护，开发者会下意识地重用已有的代码片段、遵循特定的命名规范或采用惯用的编程习语。为了验证这一假说，作者引入了自然语言处理（NLP）领域中成熟的统计语言模型，特别是 N-gram 模型，来捕捉代码中的这种局部规律性。

在实证研究环节，作者构建了极其庞大的数据集，涵盖了用作基准对比的自然语言语料（包括 Brown 语料库和 Gutenberg 语料库）以及大规模的代码语料（包括 10 个知名的 Java 开源项目如 Eclipse、Lucene，以及 Ubuntu 系统中涉及多个领域的 C 语言源代码）。研究采用“交叉熵”（Cross-Entropy）作为核心度量指标。在信息论中，交叉熵反映了模型对下一个出现的词元（Token）感到

“惊讶”的程度。如果代码具有高度的规律性，模型就能以较高的概率预测下一个词元，从而产生较低的交叉熵。

实验结果揭示了几个令人深思的现象。首先，**软件代码表现出了惊人的统计规律性，其可预测程度甚至超过了自然语言。** 数据显示，随着 N-gram 模型中 N 值的增加（即上下文窗口变大），代码的交叉熵呈现出急剧下降的趋势。在某些情况下，代码的熵值甚至降至 2 比特/词元以下，这意味在给定了前几个词元后，程序员接下来要写什么几乎是确定的。这种规律性并非仅由编程语言简单的语法结构（如 Java 相比英语更严格的语法）造成，因为即便在同一语言内部，不同项目之间的模型迁移效果也远不如项目内部自测的效果好。这一发现引出了论文的另一个关键结论：**软件的规律性具有强烈的局部性和项目特异性（Project Specificity）。** 每个软件项目都像是一个独立的“方言”区，拥有自己独特的词汇分布、命名习惯和代码风格。这意味着，通过利用项目自身的语料库进行训练，可以获得极高精度的预测模型。

为了进一步证明这种统计规律的实用价值，作者并未止步于理论计算，而是开发并部署了一个实际的 Eclipse 代码补全插件。该插件并没有利用复杂的编译器语义信息，仅仅通过简单的三元组（Trigram）统计模型来预测代码。实验对比了 Eclipse 自带的基于类型系统的补全引擎（ECSE）和融合了统计信息的补全引擎（MSE）。结果令人振奋：统计模型不仅增强了补全能力，更重要的是，它能够有效预测那些静态分析无法推断的“非语言关键字”——即开发者自定义的方法名、变量名和类名。事实上，约 50% 的有效补全建议是基于统计规律得出的项目特定词汇。这一结果强有力地证明了，代码中蕴含的“自然语义”往往通过统计上的共现关系表现出来，且这种浅层的统计特征可以被低成本地转化为高效的工程工具。

此外，论文还展望了这种基于大语料库的统计方法在未来的广阔应用前景，包括辅助残障人士进行语音编程（利用模型纠正语音识别错误）、代码摘要生成（类似机器翻译任务）以及深层代码属性挖掘。作者认为，许多深层的语义属性（如类型约束、潜在 Bug）往往会在表层的词法序列中留下统计学上的“指纹”，这为非完备的轻量级程序分析打开了大门。

### 三、对软件漏洞分析挖掘任务的思考

尽管《On the Naturalness of Software》一文主要关注代码补全等辅助开发场景，但其提出的“代码即自然语言”的哲学观及统计建模方法，对现代软件安全领域，特别是漏洞分析与挖掘任务，具有极其深远的启示。结合论文的理论框架，我们可以从以下几个维度重新审视漏洞挖掘工作：

首先，可以将漏洞挖掘视为一种“异常检测”任务，即寻找“不自然”的代码片段。论文中反复提到的交叉熵代表了“惊讶度”。基于“软件自然性”假说，绝大多数正常、安全的代码都遵循着特定的统计规律，呈现低熵状态。反之，那些包含漏洞的代码往往源于开发者对常用编程模式的误用、疏忽或逻辑混乱，这些代码在统计模型眼中极有可能是高熵的、令人“惊讶”的。例如，在 C 语言的大规模语料库中，`malloc` 函数的调用后极大概率会紧跟一个 `if (ptr == NULL)` 的空指针检查。如果统计模型学习到了这种强关联，那么当它遇到一段只分配内存却缺失检查的代码时，其预测概率会骤降，产生高交叉熵警报。这意味着，我们可以利用海量代码训练出的语言模型作为一种通用的“代码自然性检测器”，对目标代码库进行扫描。那些被模型判定为极度“反直觉”或“罕见”的代码区域，往往就是逻辑错误或安全漏洞的栖息地。这种方法相比于传统的基于规则的静态扫描，具有更好的泛化能力，能够发现未知的异常模式。

其次，利用统计推断挖掘隐式的“安全协议”与上下文约束。论文中提到了利用统计共现关系来辅助推断代码逻辑。在漏洞分析中，很多漏洞的产生是因为开发者违反了 API 使用的隐式约束（例如：在多线程环境下访问共享资源前未加锁，或在数据库查询前未清洗输入）。传统的静态分析工具（SAST）需要安全专家手动编写复杂的污点传播规则或状态机模型，这不仅耗时且难以覆盖所有边缘情况。借鉴论文的思路，我们可以让模型从已有的、经过安全验证的高质量代码库中，自动学习这些 API 调用的序列模式（Sequence Patterns）。例如，模型可能会学到，“在调用 `File.open` 之后，99% 的情况下会在同一个作用域内调用 `File.close`”。这种统计上的强相关性实际上对应了安全编程中的“资源管理协议”。一旦模型掌握了这些隐式协议，就可以自动检测出违反协议的代码路径（如资源泄漏或死锁），从而实现自动化的漏洞挖掘。这实际上是将漏洞挖掘问题转化为一个序列标注或序列分类问题。

再次，“对齐语料库”的概念为自动化漏洞修复（Automated Program

**Repair, APR) 提供了新的技术路线。** 论文在“未来展望”中提到了利用对齐语料库（如英文文档与代码的对应）进行翻译任务。在软件安全领域，版本控制系统（如 Git）中保存的海量“漏洞修复提交”（Bug-fix Commits）构成了天然的平行语料库：修改前的代码是“有漏洞的（不自然的）”，修改后的代码是“安全的（自然的）”。正如统计机器翻译模型学习如何将“错误的句子”翻译成“通顺的句子”，我们完全可以训练深度学习模型学习从“漏洞模式”到“安全模式”的映射关系。这种基于数据驱动的修复方法，不依赖于对程序语义的完全理解，而是依赖于从历史数据中提取的修复模式，对于处理常见的类型错误、缓冲区溢出或输入验证缺失等漏洞具有极高的效率。

最后，**统计视角有助于缓解静态分析中的“不确定性”难题。** 在传统的漏洞挖掘中，确定一个变量是否为“用户可控输入”（Source）往往非常困难，且容易导致误报。论文指出，代码的词汇（变量名、函数名）蕴含了丰富的语义信息。通过统计建模，我们可以发现，命名为 `user_input`、`req`、`query` 的变量，或者出现在特定 Web 框架入口处的变量，在统计上具有极高的概率是“污点源”。将这种基于自然语言统计的概率信息融合到传统的控制流图（CFG）或数据流分析中，可以大幅提高漏洞扫描工具的准确性，帮助分析人员聚焦于真正的高风险点。

## 四、 结论

《On the Naturalness of Software》一文的价值在于它打破了形式化方法的垄断，指出软件工程不仅是逻辑的构建，也是概率的统计。它证明了简单的统计模型能够捕捉到复杂的软件行为规律。对于软件漏洞分析与挖掘而言，这一视角的引入意味着我们不再仅仅依赖僵化的规则匹配，而是可以利用海量代码数据中蕴含的“群体智慧”，构建出能够感知代码“正常形态”的智能模型。当漏洞被重新定义为“统计学上的异常”时，漏洞挖掘便拥有了更广阔的自动化前景和更强大的抗干扰能力。

## 参考文献

- [1] Hindle, A., Barr, E. T., Gabel, M., Su, Z., & Devanbu, P. (2012). On the naturalness of software. *Proceedings of the 34th International Conference on Software Engineering (ICSE)*, 837-847.
- [2] Ray, B., Hellendoorn, V., Godhane, S., Tu, Z., Bacchelli, A., & Devanbu, P. (2016). On the "naturalness" of buggy code. *Proceedings of the 38th International Conference on Software Engineering (ICSE)*, 428-439.
- [3] Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2018). A survey of machine learning for big code and naturalness. *ACM Computing Surveys (CSUR)*, 51(4), 1-37.